



PROYECTO PRODUCTIVIZACIÓN

Airbnb Madrid – Price Predictor API

20 MAYO 2026

ARIELA ASTORGA
EDUARDO VALDERRAMA

DATASET

Inside Airbnb Madrid – Detailed Listings (listings.csv.gz)

25,000

LISTINGS TOTALES

79

COLUMNAS

14,157

POST LIMPIEZA

11

FEATURES SELEC.

FEATURES SELECCIONADAS

neighbourhood_cleansed | room_type | accommodates | bathrooms | bedrooms | beds | price | minimum_nights |
number_of_reviews | review_scores_rating | availability_365



EDA - PRICE ANALYSIS

Outlier detection using IQR method

€305

LÍMITE SUPERIOR IQR
(CUTOFF OUTLIER)

79

PRECIO MEDIO POST
LIMPIEZA

14,157

FILAS ELIMINADAS
COMO OUTLIERS

FÓRMULA IQR

$Q1 = €73 \mid Q3 = €162 \mid IQR = €89 \mid \text{Upper Limit} = Q3 + (1.5 \times IQR) = €305$

Cualquier listing con un precio superior a €305 fue eliminado como outlier

Listings con 0 disponibilidad y habitación Compartida / Hotel también fueron eliminados

PREPROCESAMIENTO

Limpieza, encoding y split de data

- 1) **Eliminación outliers** Precios superiores a €305 (límite superior IQR) eliminados
- 2) **Filtro Room Type** Únicamente listings Entire Home/Apartment y Private Room
- 3) **Filtro Disponibilidad** Listings con 0 disponibilidad fueron eliminados
- 4) **Agrupamiento de Barrios** 128 barrios reducidos al Top 33 + Other (mean threshold: 139 listings)
- 5) **One-hot Encoding** neighbourhood_cleansed y room_type encoded como columnas binarias
- 6) **Train / Test Split** 80/20 split – 11,613 train / 2,904 test rows – 36 features

MODELO V1 vs V2

Random Forest Regressor – el por qué cambiamos de datasets

V1 — Dataset Resumido

listings.csv (18 columnas)

MAE €35 / noche

RMSE €48 / noche

R² 0.38

Faltan key features como accommodates, bedrooms, bathrooms

V2 — Dataset Detallado

listings.csv.gz (79 columnas)

MAE €27 / noche

RMSE €37 / noche

R² 0.61

Atributos más detallados — accommodates, bedrooms, bathrooms, review scores

RESULTADOS FINALES

Random Forest Regressor – Dataset Detallado (V2)

€27

MAE (ERROR
PROMEDIO)

€37

RMSE

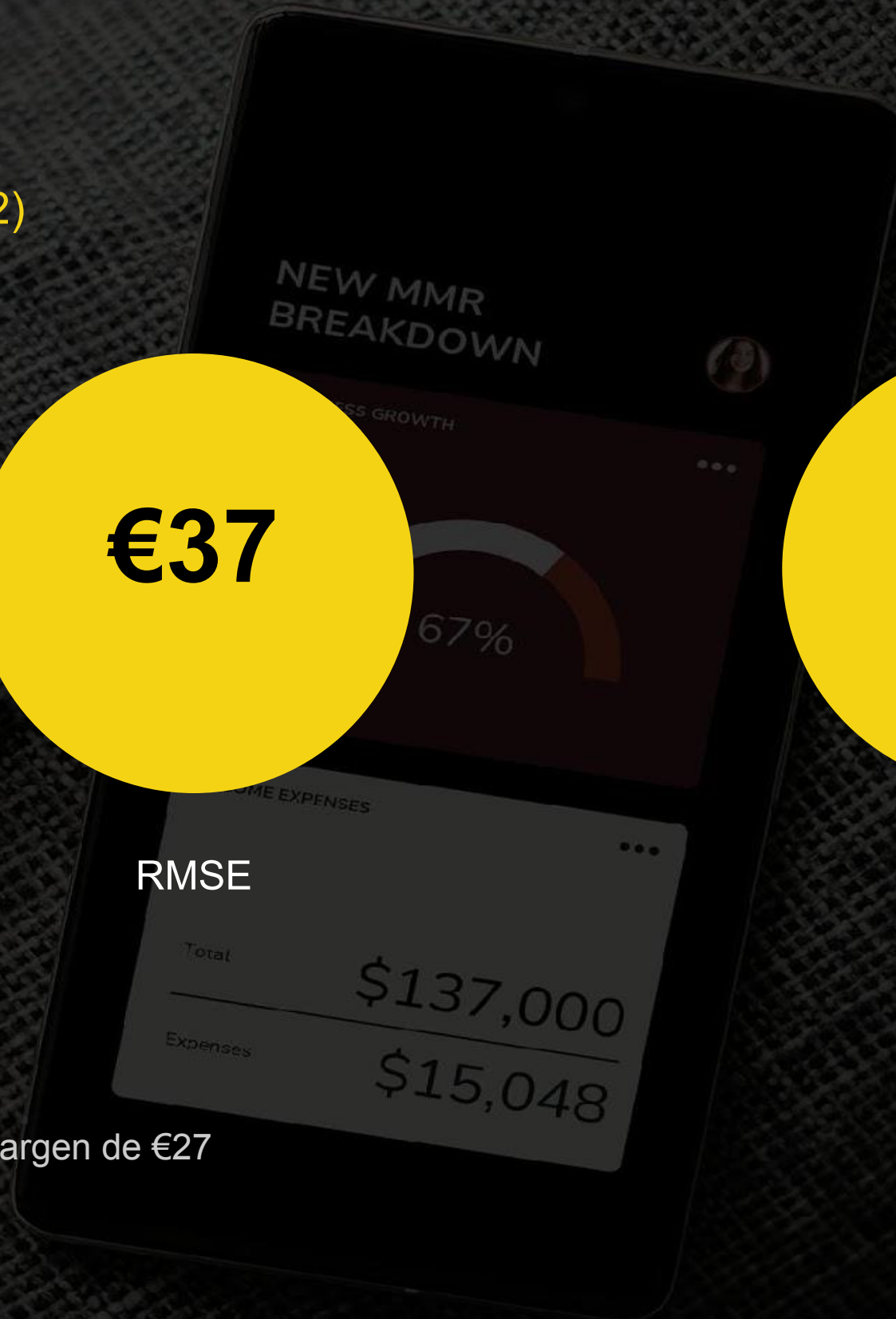
0.61

R^2

INTERPRETACIÓN

En promedio, el modelo predice el precio por noche con un margen de €27

El R^2 significa que explica el 61% de la varianza en el precio



API REST EN FLASK

ESTRUCTURA

Home - Ruta principal de la API

```
@app.route("/", methods=["GET"])
```

Imprime un mensaje de inicio:

`https://tu-api.onrender.com/` -> <https://online-ds-bridge-projects-arielaastorga.onrender.com/>

```
"message": "API Flask para predicción de precios de apartamentos",
```

```
"endpoints": ["/health", "/predict_query", "/predict"]
```

Health - Comprueba que la API está funcionando

```
@app.route("/health", methods=["GET"])
```

imprime mensaje:

```
"status": "ok",
```

```
"service": "running"
```

API REST EN FLASK

ESTRUCTURA

FUNCIÓN VERIFICA PARÁMETROS DE ENTRADA

```
def preparar_fila_entrada(data):
```

Esa función convierte y valida el diccionario de entrada para dejarlo listo para el modelo. Transforma los datos “crudos” que envía el usuario en una fila numérica, válida y con el formato exacto que necesita el modelo.

- Revisa que estén todos los campos necesarios; si falta alguno, lanza un error.
- Convierte los campos numéricos a float y da error si alguno no es numérico.
- Limpia los campos categóricos (neighbourhood_cleansed y room_type) quitando espacios y comprobando que sus valores sean válidos (**Por el one-hot encoding**)
- Completa el resto de columnas con 0 para mantener la misma estructura que esperaba el modelo.
- Activa con 1 la columna correspondiente al barrio y al tipo de habitación, como si hiciera un one-hot encoding manual.
- Devuelve todo en un DataFrame de pandas con las columnas en el orden correcto para model.predict()

API REST EN FLASK

ESTRUCTURA

Ruta con el parámetro PATH

```
@app.route("/neighbourhood/<name>", methods=["GET"])
def neighbourhood(name):
    return jsonify({"neighbourhood": name})
```

Ruta con parámetros de la QUERY - GET

Usa GET y lee parámetros de la query desde la URL: por ejemplo ?edad=30&zona=centro

Get lee, obtiene información

GET /predict-query: manda parámetros en la URL con request.args

```
@app.route("/predict-query", methods=["GET"])
def predict_query():
    data = request.args.to_dict()
    X = preparar_fila_entrada(data)
    prediction = model.predict(X)[0]
    Devuelve un json con la predicción
```


API REST EN FLASK

ESTRUCTURA

Ruta con parámetros de la QUERY - POST

Utiliza POST y los datos van en el cuerpo de la petición como JSON

, por ejemplo {"edad": 30, "zona": "centro"}

Usa post porque manda datos al servidor para que él los procese

POST /predict: mandas un JSON en el cuerpo con `request.get_json()`.

```
@app.route("/predict", methods=["POST"])
```

```
def predict():
```

Va a la función **preparar_fila_entrada(data)** y hace la predicción

DESPLIEGUE EN RENDER

Flujo de trabajo:

Se crea el servicio en Render dashboard

Conectar a la cuenta de GitHub y selecciona el repositorio de la API.

Se utilizan estos valores base:

lenguaje: Python 3,

build command: `pip install -r requirements.txt` y

start command: `gunicorn app:app`.

Branch `main` o la rama donde esté tu código

Root Directory vacío, salvo que tu app esté en una subcarpeta

Create Web Service: se descarga el repo, instala dependencias y arranca la API;

Al terminar, se obtiene la URL pública:

<https://online-ds-bridge-proyectos-arielaastorga.onrender.com/>

Se prueba el despliegue y rutas públicas:

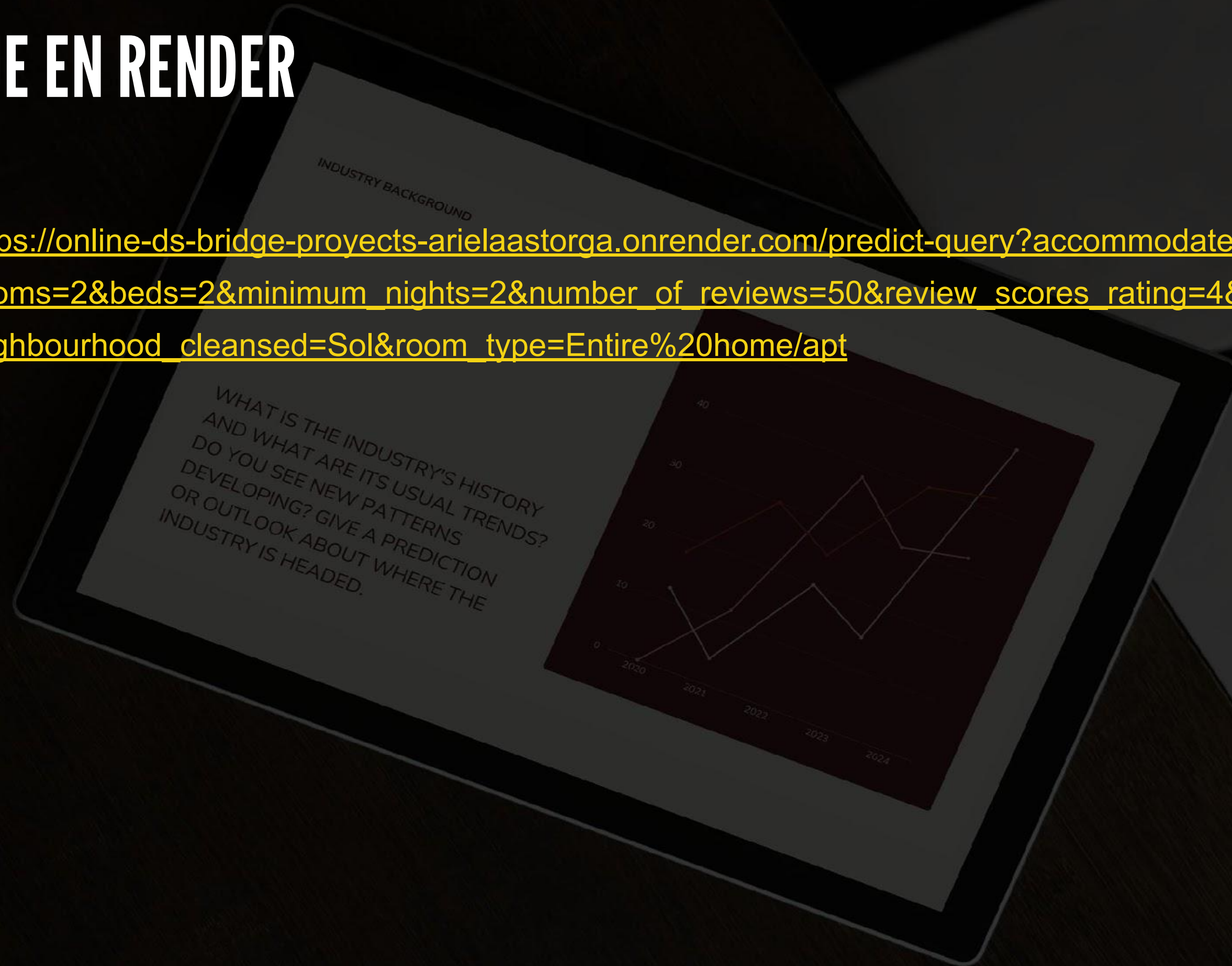
<https://online-ds-bridge-proyectos-arielaastorga.onrender.com/health>

```
{"service": "running", "status": "ok"}
```


DESPLIEGUE EN RENDER

Predicción de precio:

https://online-ds-bridge-proyectos-arielaastorga.onrender.com/predict-query?accommodates=4&bathrooms=1&bedrooms=2&beds=2&minimum_nights=2&number_of_reviews=50&review_scores_rating=4&availability_365=180&neighbourhood_cleansed=Sol&room_type=Entire%20home/apt



DESPLIEGUE EN AWS

Flujo de trabajo:

Se crea carpeta del proyecto, copio mi repo Github

Se creo carpeta de endpoints, entorno virtual, activo UV, instalo Flask, pandas y resto librerías...

Problema 1:

Problema de compatibilidad entre numpy==1.26.4 y Python 3.14

Resuelto eligiendo una **instancia t3.small**

Programas usan 3-4gb de RAM, con swap empujo las cosas a un cajón y usa un espacio en mi disco duro

Almacenamiento 8 a 20GB, y gp3 (disco duro SSD de la instancia, más rápido)

Se instala Python 3.12

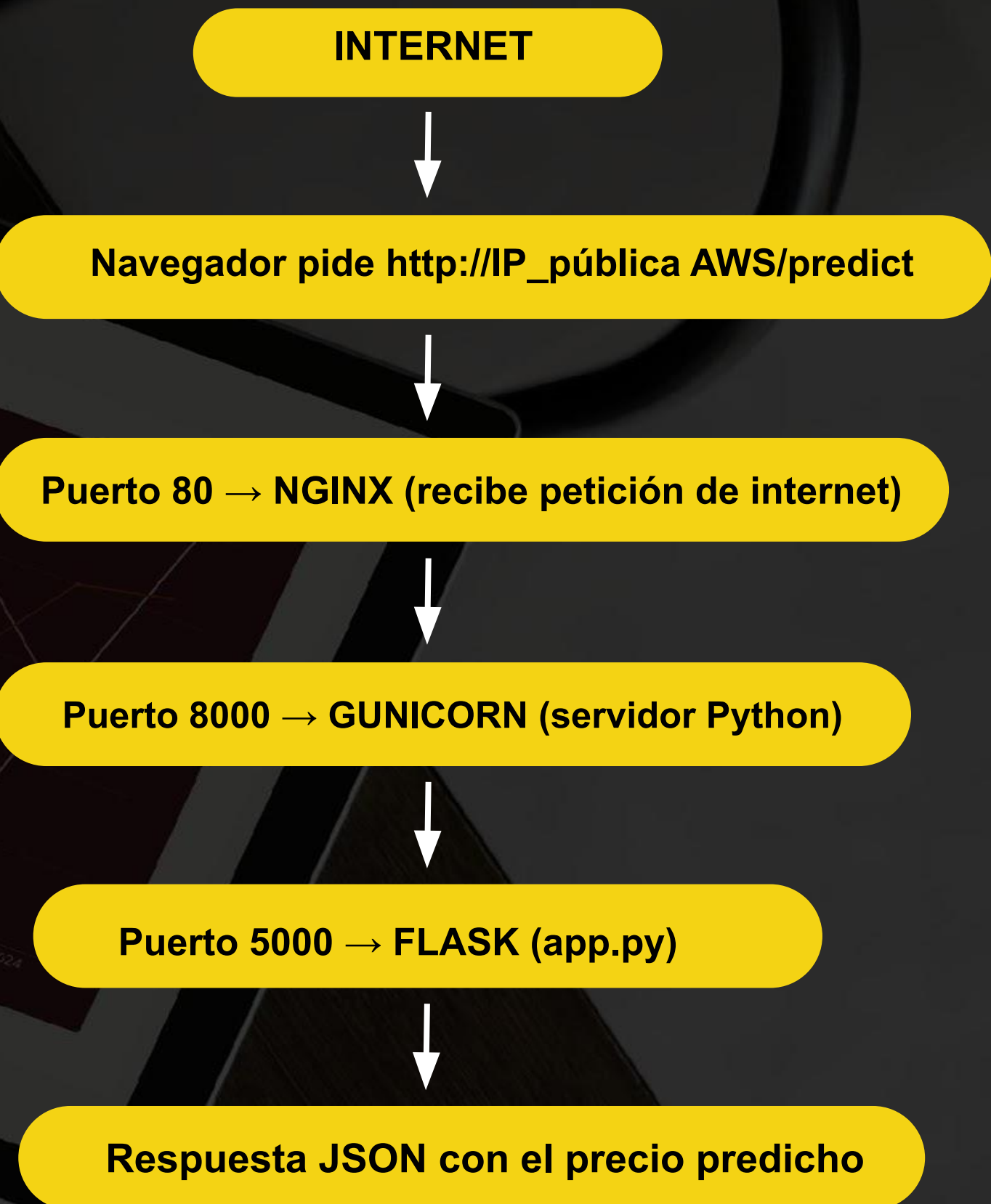
Se utiliza **Swap**, es memoria virtual extra que usa el disco duro cuando la RAM se llena. Es más lento que la RAM pero evita que el sistema se quede sin memoria durante instalaciones pesadas.

Se crean enlaces simbólicos a mi repo y trato de arrancar app.py:

Problema 2:

No arrancaba porque le faltaba al archivo app.py:

```
if __name__ == "__main__":  
    app.run(host="0.0.0.0", port=5000)
```



DESPLIEGUE EN AWS

Health check — verificar que está viva:

<http://13.60.90.39/health>

Página principal:

<http://13.60.90.39/>

Predicción GET — apartamento en Sol:

http://13.60.90.39/predict-query?neighbourhood_cleansed=Sol&room_type=Entire%20home/apt&accommodates=4&bathrooms=1&bedrooms=2&beds=2&minimum_nights=2&number_of_reviews=50&review_scores_rating=4.5&availability_365=200

Predicción GET — habitación privada en Goya:

http://13.60.90.39/predict-query?neighbourhood_cleansed=Goya&room_type=Private%20room&accommodates=2&bathrooms=1&bedrooms=1&beds=1&minimum_nights=3&number_of_reviews=30&review_scores_rating=4.2&availability_365=150



GROUND

MUCHAS GRACIAS

THE INDUSTRY'S HISTORY

40

30

20

